Intermediate (wrap real logic behind a route) — Solution

Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `chat_route_practice.py`: this is the exact shape of the Build Task's `POST /run-task` — a typed Pydantic request in, real logic in the middle, a typed reply out, and every failure deliberately mapped to the right status code. Practicing it on a tiny fake chat function, not a real agent, keeps the focus purely on the route's error handling, not the logic behind it. **If not:** the Build Task would be the first place you ever mapped errors to 400 vs 500, with a multi-agent pipeline making every mistake harder to isolate.

`run_chat()` below is a small stand-in for Doc04's real chat function — swap in the real one, the route around it doesn't change.

Solution

Story: a client needs to know whether to fix its own request or just try again later — a single generic 500 status for every kind of failure can't communicate that distinction at all. A named `ChatInputError`, caught and mapped to a 400, marks "your input specifically is the problem" on purpose. **If not:** an empty message and a genuine server crash would look identical to the client, and it would keep resending a malformed request that can never succeed.

```
# practice/chat_route_practice.py
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel

app = FastAPI()

class ChatRequest(BaseModel):
    message: str

class ChatResponse(BaseModel):
    reply: str

class ChatInputError(Exception):
    """Raised on purpose when input is well-formed but not usable."""
    pass
```

```
def run_chat(message: str) -> str:
    # why: "" passes Pydantic's check (it IS a str), so this rule
    # has to live in our own code
    if not message.strip():
        raise ChatInputError("message cannot be empty")
    return "You said: " + message

@app.post("/chat", response_model=ChatResponse)
def chat(request: ChatRequest) -> ChatResponse:
    try:
        reply_text = run_chat(request.message)
        return ChatResponse(reply=reply_text)
    except ChatInputError as exc:
        # how: our own message is safe to show — we wrote it
        raise HTTPException(status_code=400, detail=str(exc))
    except Exception:
        raise HTTPException(status_code=500, detail="Something went wrong.")
```

Expected output for `{"message": ""}`:

```
400 {"detail":"message cannot be empty"}
```

Expected output for `{"message": "hi"}`:

```
200 {"reply":"You said: hi"}
```